

Machine Learning with TensorFlow

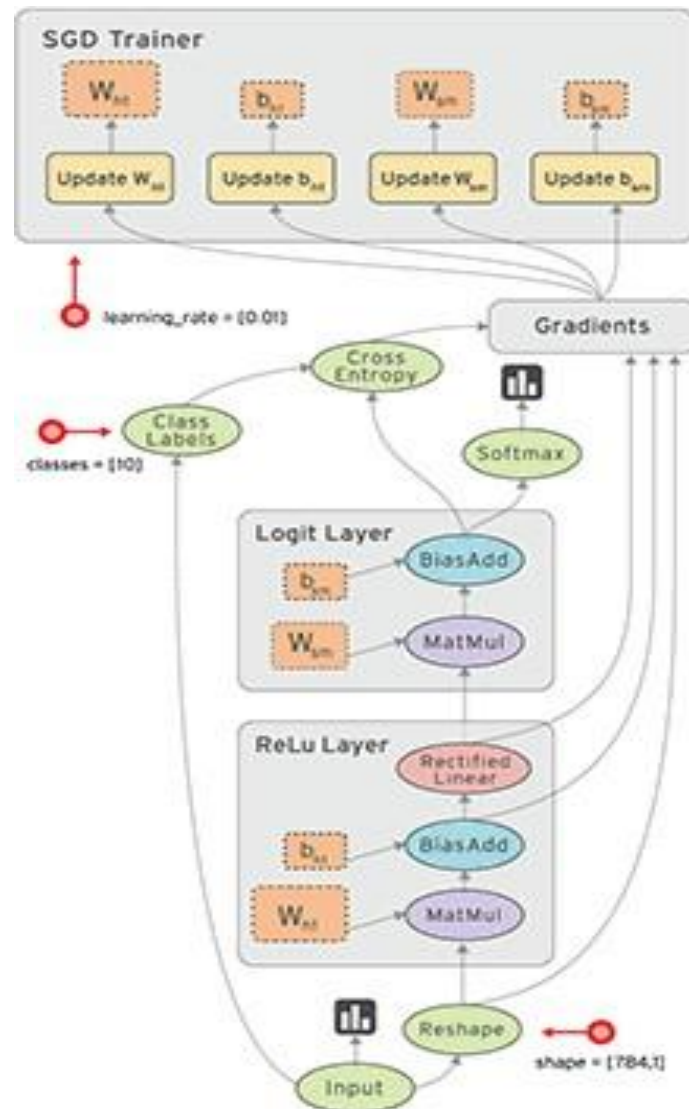
Deep Learning with TensorFlow

MACHINE LEARNING WITH TENSORFLOW

👉 실습

● TensorFlow

- TensorFlow is an open source software library for numerical computation using data flow graphs.
- Data flow graph는 **Node**와 **Edge**로 구성된 방향성 있는 그래프로 표현
- Node는 mathematical operation 데이터의 입출력 작업을 수행
- Edge는 동적 크기의 다차원배열(Tensor)을 Node로 실어 나르는 역할



MACHINE LEARNING WITH TENSORFLOW

👉 실습

- 아래의 코드를 이용하여 Hello World 출력

- Node는 mathematical operation, 데이터의 입출력 작업을 수행.

```
import tensorflow as tf  
  
node = tf.constant("Hello World")  
  
sess = tf.Session()  
  
print(sess.run(node).decode())
```

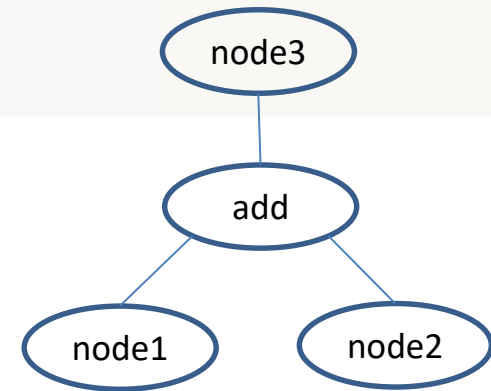
```
Hello World
```

<http://stackoverflow.com/questions/6269765>

MACHINE LEARNING WITH TENSORFLOW

👉 실습

● 간단한 수학 연산 수행



```
import tensorflow as tf

node1 = tf.constant(10, dtype=tf.float32)
node2 = tf.constant(20, dtype=tf.float32)

node3 = tf.add(node1, node2) # node3 = node1 + node2

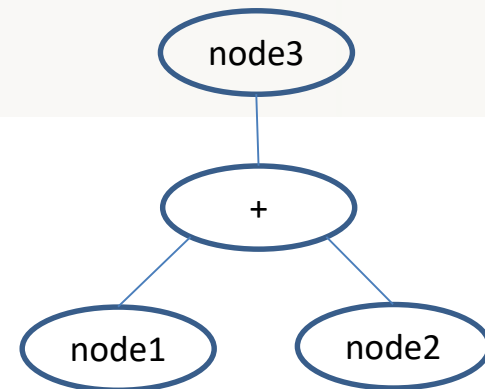
sess = tf.Session()

# print(sess.run([node1, node2]))
print(sess.run(node3))
```

```
[10.0, 20.0]
30.0
```

MACHINE LEARNING WITH TENSORFLOW

👉 실습



- placeholder 사용 그래프를 실행시키는 단계에서 값을 던져 줌

```
# placeholder 사용
# 2개의 수를 입력으로 받아서 덧셈 연산
import tensorflow as tf

node1 = tf.placeholder(dtype=tf.float32)
node2 = tf.placeholder(dtype=tf.float32)

node3 = node1 + node2

sess = tf.Session()

result = sess.run(node3, feed_dict={ node1 : 10, node2 : 20})
# result = sess.run(node3, feed_dict={ node1 : input(), node2 : input()})
# result = sess.run(node3, feed_dict={ node1 : [1,2,3], node2 : [10,20,30]})

print("덧셈연산결과 : {}".format(result))
```

MACHINE LEARNING WITH TENSORFLOW

☞ Supervised Learning Type(1)

● Linear Regression

➤ 7시간 공부하면 0 ~ 100점 중 몇 점을 받을 수 있는가?

x(시간)	y(점수)
10	95
8	80
5	68
2	15
1	5

MACHINE LEARNING WITH TENSORFLOW

☞ Supervised Learning Type(2)

● Logistic Regression (Binary Classification)

➤ 7시간 공부하면 합격인가 혹은 불합격 인가? (2개 중 1개 , 0 or 1)

x(시간)	y(결과)
10	PASS(1)
8	PASS(1)
5	FAIL(0)
2	FAIL(0)
1	FAIL(0)

MACHINE LEARNING WITH TENSORFLOW

☞ Supervised Learning Type(3)

● Multinomial Classification

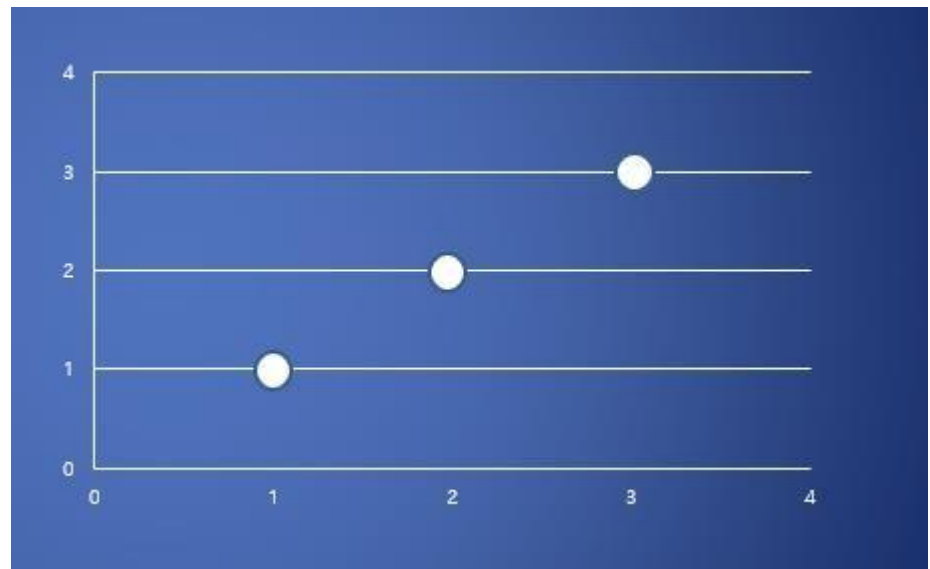
➤ 7시간 공부하면 어떤 grade를 받을 수 있는가? (A,B,C,D,F – N개 중 1개)

x(시간)	y(결과)
10	A
8	B
5	D
2	F
1	F

MACHINE LEARNING WITH TENSORFLOW

👉 Linear Regression – 사용할 training data set

x	y
1	1
2	2
3	3



MACHINE LEARNING WITH TENSORFLOW

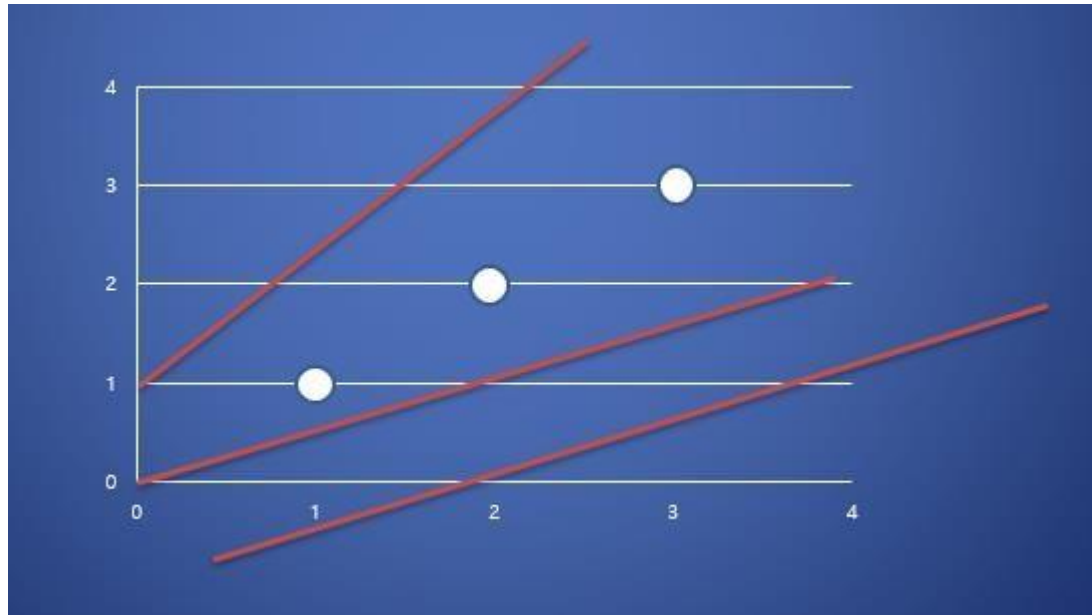
☞ Linear Hypothesis

- 많은 데이터와 현상들이 linear한 형태를 가진다.
 - 일반적으로,
 - 많은 시간 공부를 하면 점수를 더 많이 받고
 - 오랜 시간 일을 하면 더 많은 돈을 벌고
 - 공을 세게 던지면 던질수록 더 멀리 나아가고
 - 배달지역이 멀수록 배달시간이 오래 걸린다.
- 우리 예제도 이러한 linear한 형태라고 가정하고 하나의 가설(Hypothesis)을 세운다.

MACHINE LEARNING WITH TENSORFLOW

☞ Linear Hypothesis

- Linear Hypothesis를 정의하는 것은 training data set에 잘 맞는 linear한 선을 긋는 것으로 생각할 수 있다.
- Hypothesis를 수정해 나가면서 데이터에 가장 적합한 선을 찾는 과정이 바로 학습.



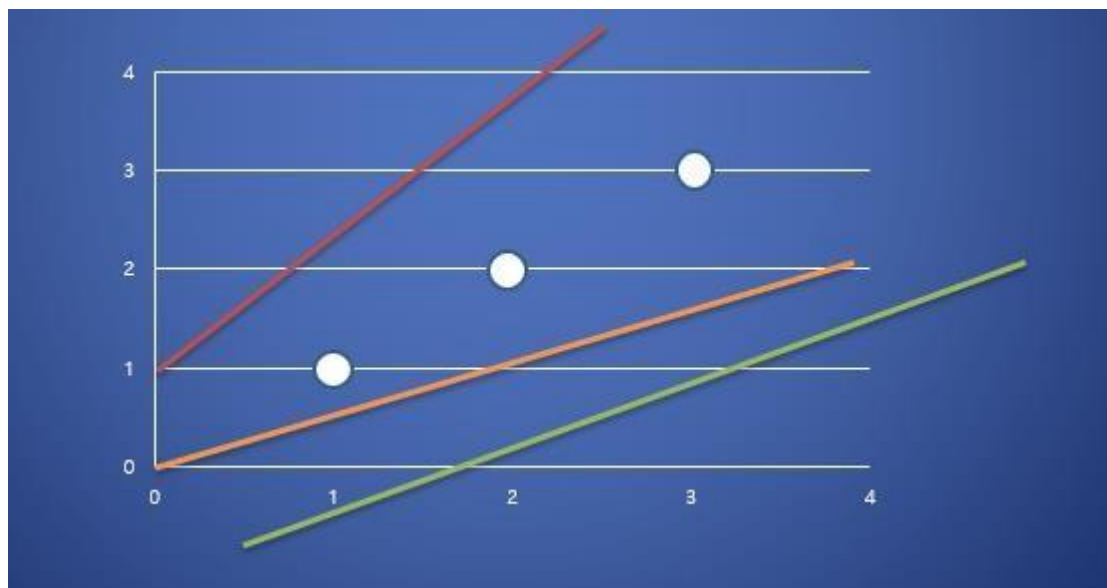
MACHINE LEARNING WITH TENSORFLOW

☞ Linear Hypothesis

- 이러한 선들을 수학적으로 표현하면 다음과 같다.

$$H(x) = Wx + b$$

Hypothesis(가설)

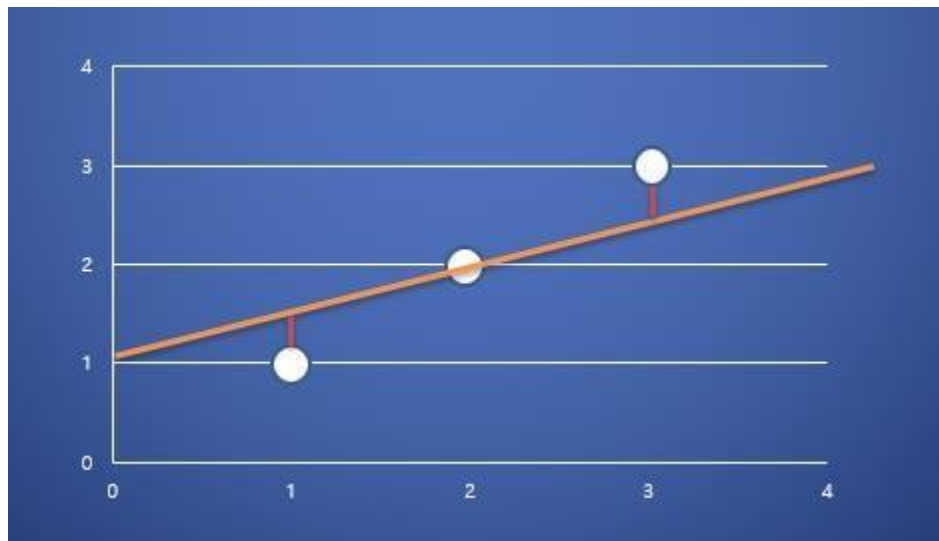


MACHINE LEARNING WITH TENSORFLOW

☞ Linear Hypothesis

- 우리가 원하는 것은 training data set에 가장 적합한 W 와 b 의 값을 찾는 것.
 - 가장 적합한 W 와 b 의 값을 찾으려면 **어떻게** 해야 하는가?
 - Cost(Loss) Function 사용 - 최소제곱법

$$H(x) = Wx + b$$

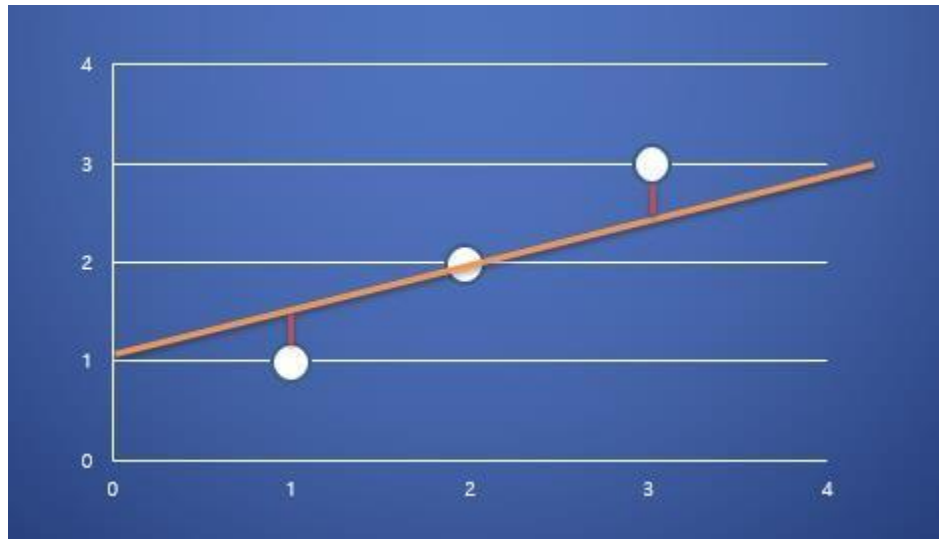


MACHINE LEARNING WITH TENSORFLOW

☞ Cost(Loss) Function

- 가설과 training data간의 값의 차를 계산하는 함수 $H(x) = Wx + b$

$$\frac{(H(x^{(1)}) - y^{(1)})^2 + (H(x^{(2)}) - y^{(2)})^2 + (H(x^{(3)}) - y^{(3)})^2}{3}$$



MACHINE LEARNING WITH TENSORFLOW

☞ Cost(Loss) Function

- 가설과 training data간의 값의 차를 계산하는 함수

$$\frac{(H(x^{(1)}) - y^{(1)})^2 + (H(x^{(2)}) - y^{(2)})^2 + (H(x^{(3)}) - y^{(3)})^2}{3}$$



$$cost(W, b) = \frac{1}{n} \sum_{i=1}^n (H(x^{(i)}) - y^{(i)})^2$$

☞ Cost(Loss) Function

- 결국 Cost function의 값을 **최소**로 만드는 W 와 b 의 값을 찾아내는 것이 **학습**의 목표

$$cost(W, b) = \frac{1}{n} \sum_{i=1}^n (H(x^{(i)}) - y^{(i)})^2$$

MACHINE LEARNING WITH TENSORFLOW

👉 Tensors

- Types은 Tensor의 데이터 타입을 의미, Type 변경 가능

```
# Tensor의 type변경
import tensorflow as tf
import numpy as np

node1 = tf.constant([1,2,3], dtype=tf.int32)

node2 = tf.cast(node1,dtype=tf.float32)
# node2 = tf.cast(node1,dtype=np.float64)

sess = tf.Session()

print(sess.run(node1))
print(sess.run(node2))
```

MACHINE LEARNING WITH TENSORFLOW

👉 Linear regression 실습 (1)

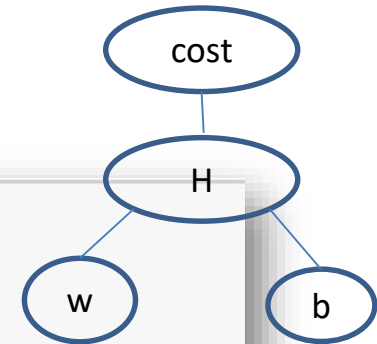
```
## Linear Regression 실습
import tensorflow as tf

# training data set
x = [1,2,3]
y = [1,2,3]

# Weight & bias
W = tf.Variable(tf.random_normal([1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")

# Hypothesis
H = W * x + b

# Cost function
cost = tf.reduce_mean(tf.square(H - y))
```



MACHINE LEARNING WITH TENSORFLOW

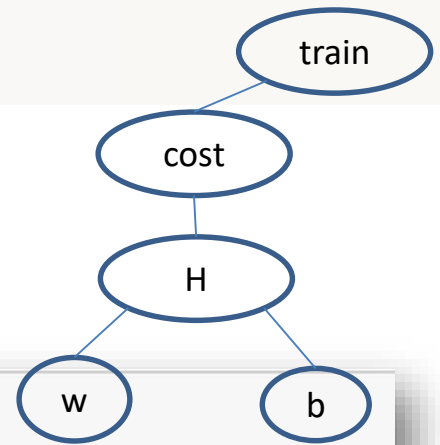
👉 Linear regression 실습 (2)

```
# cost function minimize
optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.01)
train = optimizer.minimize(cost)

# runner 생성
sess = tf.Session()

# Variable 초기화
sess.run(tf.global_variables_initializer())

for step in range(3000):
    _, w_val, b_val, cost_val = sess.run([train, W, b, cost])
    if step % 300 == 0:
        print("W의 값 : {}, b의 값 : {}, cost의 값 : {}".format(w_val, b_val, cost_val))
```



MACHINE LEARNING WITH TENSORFLOW

👉 Linear regression 실습 - **placeholder**를 이용하여 초기데이터 세팅

```
## Linear Regression 실습
```

```
import tensorflow as tf
```

```
# training data set
```

```
x = tf.placeholder(dtype=tf.float32)
```

```
y = tf.placeholder(dtype=tf.float32)
```

```
x_data = [1,2,3]
```

```
y_data = [1,2,3]
```

```
#####
```

```
# 중간코드 생략
```

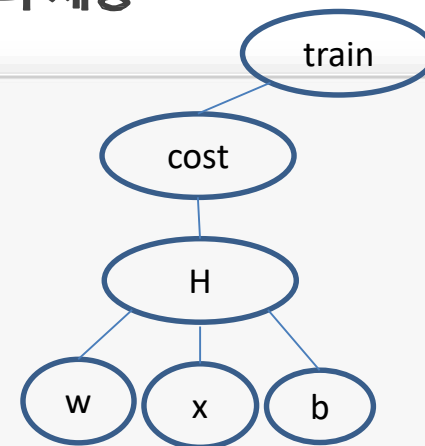
```
#####
```

```
for step in range(3000):
```

```
    _, w_val, b_val, cost_val = sess.run([train, W, b, cost],  
                                          feed_dict={x:x_data,y:y_data})
```

```
    if step % 300 == 0:
```

```
        print("W의 값 : {}, b의 값 : {}, cost의 값 : {}".format(w_val,b_val,cost_val))
```



MACHINE LEARNING WITH TENSORFLOW

👉 학습 종료 후 데이터를 이용한 추정

```
#####  
# 상위코드 생략  
#####  
  
for step in range(3000):  
    _, w_val, b_val, cost_val = sess.run([train, W, b, cost],  
                                         feed_dict={x:x_data,y:y_data})  
  
    if step % 300 == 0:  
        print("W의 값 : {}, b의 값 : {}, cost의 값 : {}".format(w_val,b_val,cost_val))  
  
## prediction  
  
print(sess.run(H,feed_dict={x:[17]}))  
  
print(sess.run(H,feed_dict={x:[100,200,300]}))
```

MACHINE LEARNING WITH TENSORFLOW

☞ Cost(Loss) function 심화

- 목적은 Cost function을 최소화 시키는 W와 b의 값을 찾는 것.

```
# Weight & bias
W = tf.Variable(tf.random_normal([1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")

# Hypothesis
H = W * x + b

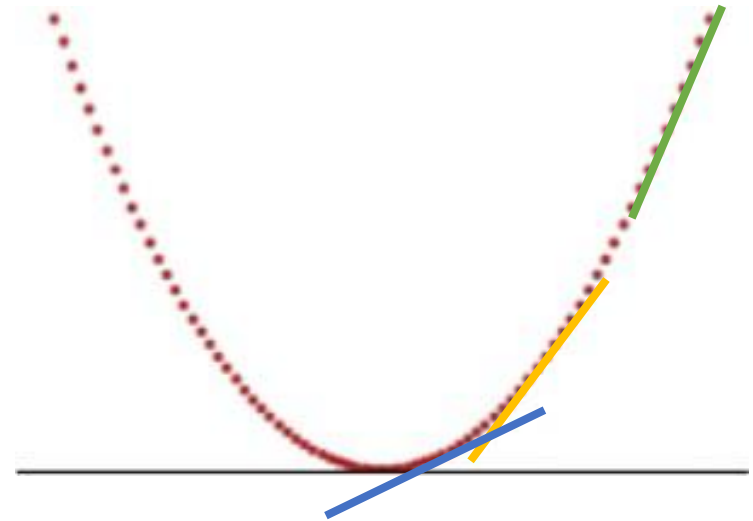
# Cost function
cost = tf.reduce_mean(tf.square(H - y))

# cost function minimize
optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.01)
train = optimizer.minimize(cost)
```

MACHINE LEARNING WITH TENSORFLOW

☞ Cost function의 값을 최소화 시키는 W 를 어떻게 알아낼 수 있는가?

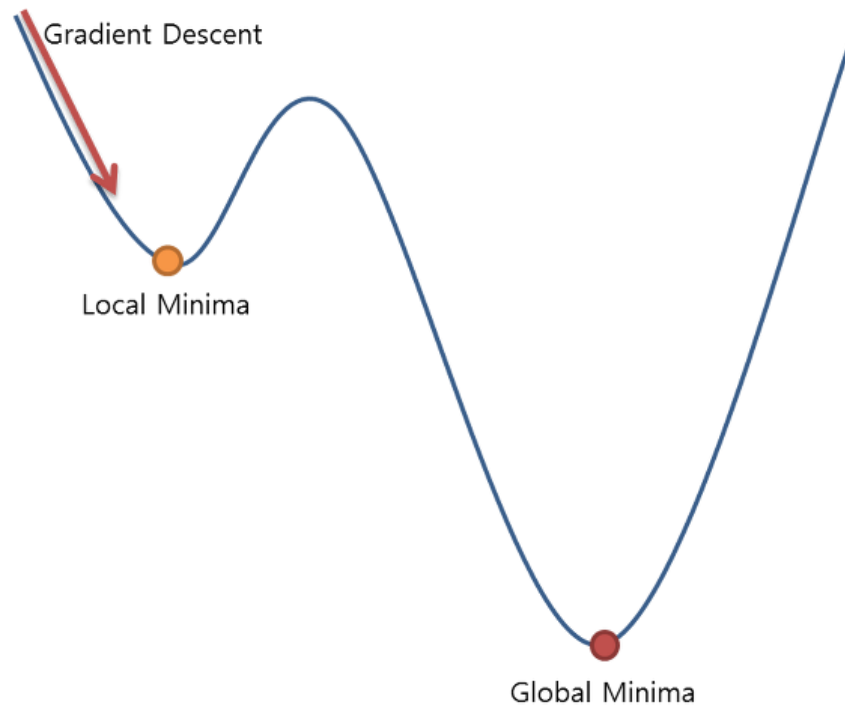
- Gradient Descent Algorithm을 이용
- 최소값을 찾기 위해 많이 사용되는 대표적인 Algorithm
- 미분을 이용



MACHINE LEARNING WITH TENSORFLOW

☞ Gradient Descent Algorithm의 예외 경사하강알고리즘

- Gradient Descent Algorithm은 항상 사용할 수 있는 것은 **아니다**.



MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

- 여러 개의 input을 이용한 linear regression

- 3번의 quiz 점수를 기반으로 시험성적 예측

x1(quiz1)	x2(quiz2)	x3(quiz3)	y(exam)
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

- Hypothesis

- 변수가 1개일때의 Hypothesis

$$H(x) = Wx + b$$

- 변수가 3개일때의 Hypothesis

$$H(x_1, x_2, x_3) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

● Matrix를 이용한 Hypothesis

$$H(x_1, x_2, \dots, x_n) = \underline{w_1x_1 + w_2x_2 + \dots + w_nx_n} + b$$


$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1w_1 + x_2w_2 + x_3w_3)$$


$$H(x) = XW + b$$

Matrix를 사용할 경우
관용적으로 대문자 사
용, X를 앞에 위치

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

- 입력으로 여러 개의 행이 존재할 경우는 어떻게 하는가?

$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1w_1 + x_2w_2 + x_3w_3)$$

x1(quiz1)	x2(quiz2)	x3(quiz3)	y(exam)
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

하나의 행
(instance)

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression

● 입력으로 여러 개의 행이 존재할 경우는 어떻게 하는가?

➢ Matrix의 곱셈을 활용

x1	x2	x3	y
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

$$\begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \\ x_{41} & x_{42} & x_{43} \\ x_{51} & x_{52} & x_{53} \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} x_{11}w_1 + x_{12}w_2 + x_{13}w_3 \\ x_{21}w_1 + x_{22}w_2 + x_{23}w_3 \\ x_{31}w_1 + x_{32}w_2 + x_{33}w_3 \\ x_{41}w_1 + x_{42}w_2 + x_{43}w_3 \\ x_{51}w_1 + x_{52}w_2 + x_{53}w_3 \end{pmatrix}$$

$[5, 3] \quad [3, 1] \quad [5, 1]$

☞ Multi-variable(multiple) linear regression

- 기본이론

$$H(x) = Wx + b$$

- 실제구현

$$H(x) = XW + b$$

MACHINE LEARNING WITH TENSORFLOW

👉 Multi-variable(multiple) linear regression 실습

- 사용데이터

x1(quiz1)	x2(quiz2)	x3(quiz3)	y(exam)
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습

```
# multiple linear regression 예제
import tensorflow as tf
import numpy as np
import pandas as pd

# training data set
x_data = [[73,80,75],
          [93,88,93],
          [89,91,90],
          [96,98,100],
          [73,66,70]]
y_data = [[152],[185],[180],[196],[142]]

# placeholder
X = tf.placeholder(shape=[None,3], dtype=tf.float32)
Y = tf.placeholder(shape=[None,1], dtype=tf.float32)

# Weight & bias
W = tf.Variable(tf.random_normal([3,1]), name="weight")
b = tf.Variable(tf.random_normal([1]), name="bias")
```


MACHINE LEARNING WITH TENSORFLOW

☞ Multi-variable(multiple) linear regression 실습

```
# Hypothesis
H = tf.matmul(X,W) + b

# Cost function
cost = tf.reduce_mean(tf.square(H-Y))

# train
train = tf.train.GradientDescentOptimizer(learning_rate=1e-5).minimize(cost)

# Session & 초기화
sess = tf.Session()
sess.run(tf.global_variables_initializer())

# 학습
for step in range(3000):
    _, cost_val = sess.run([train,cost], feed_dict={X:x_data, Y:y_data})
    if step % 300 == 0:
        print("cost : {}".format(cost_val))
```